

Database Design and ERD

A Venue Booking System requires three core entities – **Venue**, **Event**, and **Booking** – with clear relationships. A simple ERD might include:

- **Venue**: holds VenueID (PK), VenueName, Location, Capacity, and ImageUrl.
- **Event**: holds EventID (PK), EventName, Description, StartDate, EndDate, (optionally VenueID). Events can be created before a venue is assigned, so VenueID can be nullable until booking.
- **Booking**: associative linking table with BookingID (PK), EventID (FK to Event), VenueID (FK to Venue), and BookingDate. A booking ties one Event to one Venue on a specific date.

In the ERD, **Venue-Booking** is one-to-many and **Event-Booking** is one-to-many. To enforce data integrity, foreign keys should be set with **no cascade delete** (or explicitly ON DELETE RESTRICT) so that a Venue or Event with existing bookings cannot be deleted. This matches the requirement to “restrict deletion of venues or events associated with existing bookings.” Uniqueness or overlap checks must be handled in logic (e.g. ensure no two bookings for the same venue overlap in date). The key fields (VenueID, EventID, BookingID) would be unique identifiers (e.g. GUIDs or identity ints) as specified. This covers all required attributes (names, locations, capacity, dates, booking IDs, etc.) without over-normalization.

ASP.NET Core MVC Application Setup

We will scaffold an **ASP.NET Core MVC** project (e.g. via `dotnet new mvc`). For each table above, we create a corresponding model class and DbContext in Entity Framework Core. For example:

```
public class Venue {  
    public int VenueID { get; set; }  
    [Required] public string VenueName { get; set; }  
    public string Location { get; set; }  
    public int Capacity { get; set; }  
    public string ImageUrl { get; set; }  
    public ICollection<Booking> Bookings { get; set; }  
}
```

Similarly define **Event** and **Booking** classes with navigation properties. Use EF Core Code-First migrations to generate the database schema. In `Startup / Program.cs`, configure the DbContext (e.g. `options.UseSqlServer(connectionString)`) and, if user authentication is needed later, add ASP.NET Identity with roles.

For each entity, generate MVC Controllers and Views (CRUD). Using scaffolding, create **VenueController**, **EventController**, and **BookingController** with actions for Create/Read/Update/Delete. Views (Razor pages) will display lists (index), forms, and details. Implement basic validation on models (e.g. `[Required]`, `capacity > 0`). In the index/list pages, add search and filter UI: for example, include a search textbox that

submits to the controller's Index action, where we filter via a LINQ query (e.g. `db.Venues.Where(v => v.VenueName.Contains(searchString))`) ¹. This provides the required "search and filtering capabilities" to easily view venues, events, or bookings.

Role-Based Access: Since only authorized personnel (booking specialists) will use the system, secure the controllers with ASP.NET Core Authorization. For instance, decorate the admin controllers with `[Authorize(Roles = "Administrator")]` to restrict access ². (Future integration could use Azure AD or Identity roles and policies.)

Handling Images and Uploads

Each Venue (and potentially Event) has an associated image URL. In Phase 1 we use **placeholder images** (public sample URLs) and simply store the URL in the `ImageUrl` field. Later we can replace this by storing images in Azure Blob Storage. In a full implementation, the user would upload an image file (via an `<input type="file">` in a form), the server would receive an `IFormFile`, and then save it to a blob container. For now, ensure the model has an `ImageUrl` string, and display it in views (e.g. ``).

When implementing real uploads, we would protect the blob container. One approach is to keep container access private and generate time-limited SAS URLs for each image on demand ³. Short-lived SAS tokens ensure secure reads without making the container public. Alternatively, one could use Azure AD authentication to serve images. (If images can be public, another option is to obfuscate filenames and allow public read access for the images container ⁴.) The key is to use secure links and not expose admin-level credentials in the app.

Preventing Double Bookings and Ensuring Data Integrity

To **prevent double-booking**, implement logic in the Booking creation flow. Before saving a new Booking, check if any existing booking for that Venue overlaps the Event's start/end date. For example, query the `Bookings` table for the same `VenueID` where existing `StartDate <= new.EndDate` AND existing `EndDate >= new.StartDate`. If any record is found, disallow the booking as conflicting ⁵ ⁶. This ensures that no two events at one venue overlap in time. Also, in the database we could add a unique index on (VenueID, BookingDate) if bookings are one-per-day, but if events span multiple days, handle it in code as above. Use EF transactions or row-locking if concurrent bookings could occur, to avoid race conditions.

Similarly, when deleting a Venue or Event, first check for existing Bookings (e.g. `if (db.Bookings.Any(b => b.VenueID == id)) { /* refuse delete */ }`) to enforce the "restrict deletion" rule. You can also rely on the FK constraint (ON DELETE RESTRICT) which will raise an exception if you try to delete a parent with children. In that case catch the exception and show a user-friendly message. Either way, this approach aligns with best practice to "check if [the] data exists before delete action" rather than catching SQL errors ⁷.

Azure Deployment (App Service and SQL Database)

For hosting, we use **Azure App Service** for the web app and **Azure SQL Database** for data. For example, in the Azure Portal one can **create a new Web App**: choose a unique name (e.g. EventEaseApp), select the .NET runtime (e.g. .NET 8 or 10 LTS) and an OS (Windows or Linux), and choose a pricing tier (Basic or Standard) ⁸. This implicitly creates an App Service Plan and resource group. Next, on the Database tab of the Web App creation (or separately in Portal), create an Azure SQL Database: specify a server (or create new), admin credentials, and a name (e.g. EventEaseDB) ⁹. Ensure the server's firewall allows the App Service (or enable "Allow Azure services").

Alternatively, you can use Visual Studio's Publish wizard. In Visual Studio, right-click the project and choose Publish→Azure→Azure App Service. You can create a new App Service instance right from there ¹⁰. Visual Studio will even detect the need for a SQL Database (in the Service Dependencies pane) and offer to create it. When configuring the database, one enters a new DB name and admin login/password ⁹ ¹¹. The wizard then adds the connection string to the app's settings. (Best practice is to use different SQL credentials for the app vs. the admin user ¹¹.)

After creating the Azure SQL resource, update the app's `appsettings.json` or Azure Configuration Connection Strings with the new Azure SQL connection string (found in the portal under "Settings > Connection strings"). Then deploy the app: you can use Visual Studio's Publish, or use `dotnet publish` plus Azure CLI, or set up GitHub Actions. In any case, the app runs on Azure and connects to the cloud database.

Data Migration and Backup Strategy

To **migrate existing local data** into Azure SQL, you can use Entity Framework migrations (running `dotnet ef database update` against the Azure connection) if your local DB is simple. For more robust migration, export a BACPAC of the local database and import it into Azure. For example, in Azure Portal's SQL Database page use the "Export" feature to save a BACPAC to Azure Storage ¹², then use "Import" to create a new DB in Azure from that file. Azure Data Migration Service (DMS) is another option for larger schemas.

Once data is in Azure, **automated backups** are managed by the platform. Azure SQL Database automatically takes weekly full backups, differential backups every 12-24 hours, and transaction log backups roughly every 10 minutes ¹³. These allow point-in-time restore within the retention period. By default, Azure uses geo-redundant storage for backups, protecting against regional outages ¹⁴. For long-term retention (archival), you can enable LTR (Long-Term Retention) up to 10 years. For images and other files (if we later use Azure Blob Storage), we would rely on Geo-redundancy of the storage account or implement snapshot/versioning policies as needed. In all cases, having **daily backups and multi-region redundancy** ensures booking data is safely preserved in the cloud ¹³ ¹⁴.

Future Scalability and Cloud Integration

EventEase plans to integrate additional cloud services as it grows. To support scalability, we should design the app for statelessness and pluggable services. For example, user authentication can later be hooked to Azure Active Directory, and images served from Azure CDN. We can containerize parts of the system or migrate to Azure Kubernetes Service (AKS) if needed for high throughput. Caching (e.g. Azure Redis Cache) can improve performance for frequently requested data (like venue availability). Eventually, the same database model can be scaled out using Azure SQL Elastic Pools or Cosmos DB if global distribution is required. In sum, the Phase 1 build (MVC app + Azure SQL) lays the foundation, and since all components are on Azure we can add services (Key Vault for secrets, Redis for cache, Functions for background jobs, etc.) seamlessly as EventEase's needs evolve.

Sources: We follow Azure best practices for deployment and backups ⁸ ¹³, implement ASP.NET Core MVC security features (e.g. `[Authorize(Roles="Administrator")]`) ², and use standard Entity Framework Core patterns for filtering and data integrity ⁵ ¹. These ensure the platform is robust, secure, and scalable for EventEase's current and future requirements.

¹ Part 7, add search to an ASP.NET Core MVC app | Microsoft Learn

<https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/search?view=aspnetcore-10.0>

² Role-based authorization in ASP.NET Core | Microsoft Learn

<https://learn.microsoft.com/en-us/aspnet/core/security/authorization/roles?view=aspnetcore-10.0>

³ ⁴ asp.net core webapi - Efficient and Secure Access to Images in Azure Blob Storage for a List of Products - Stack Overflow

<https://stackoverflow.com/questions/79235160/efficient-and-secure-access-to-images-in-azure-blob-storage-for-a-list-of-product>

⁵ ⁶ mysql - Find booking overlaps to check dates availability - Stack Overflow

<https://stackoverflow.com/questions/25549765/find-booking-overlaps-to-check-dates-availability>

⁷ How to prevent deletion of entity if another entities has foreign key reference to it? - Stack Overflow

<https://stackoverflow.com/questions/75620066/how-to-prevent-deletion-of-entity-if-another-entities-has-foreign-key-reference>

⁸ Deploy ASP.NET Core and Azure SQL Database app - Azure App Service | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/app-service/tutorial-dotnetcore-sqldb-app>

⁹ ¹⁰ ¹¹ Publish an ASP.NET Core app to Azure with Visual Studio | Microsoft Learn

<https://learn.microsoft.com/en-us/aspnet/core/tutorials/publish-to-azure-webapp-using-vs?view=aspnetcore-10.0>

¹² Export a Database to a BACPAC File - Azure SQL Database & Azure SQL Managed Instance | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/azure-sql/database/database-export?view=azuresql>

¹³ ¹⁴ Automatic, Geo-Redundant Backups - Azure SQL Database | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/azure-sql/database/automated-backups-overview?view=azuresql>